

Learning Memory System

Condensed Study Notes

Generated: 2026-05-24 23:50 UTC

COMPUTER ARCHITECTURE AND ORGANIZATION

Imagine you're building a house. You need a blueprint that shows where all the rooms, doors, and windows go, and you also need to know how all the different parts of the house, like the plumbing and electrical systems, connect and work together. Computer architecture and organization is similar, but for computers!

At its core, it's about how computers are designed and how their different parts work together to get things done.

Think of it this way:

- **Architecture** is like the overall plan or the "what" of a computer. It defines what the computer can do and how it looks from a programmer's point of view. It's like deciding you want a house with 3 bedrooms, 2 bathrooms, and a big kitchen.
- **Organization** is like the "how" – the actual implementation details that make the architecture work. It's about how the components are physically connected and how they communicate. This is like deciding which type of pipes to use for plumbing and how to wire the electricity.

So, computer architecture is about the features that are visible to a programmer, like the instruction set (the commands the computer understands), the number of bits used to represent data, and the memory addressing capabilities. Computer organization, on the other hand, deals with the hardware components and their interconnections, such as the control signals, memory technology, and the interface between the computer and its peripherals.

COMPUTER MEMORY SYSTEM

Just like your house has different places to store things – a pantry for food, a closet for clothes, a bookshelf for books – a computer also has a system for storing information. This is called the **computer memory system**. It's where the computer keeps all the instructions it needs to run programs and all the data it's working with.

Think of it like a vast filing cabinet. Some drawers are super quick to access (like your desk drawer where you keep important papers handy), while others take a bit longer to get to (like a storage closet in the basement). The computer memory system is organized in a similar way, with different types of storage that have different speeds and capacities.

Here's a simple way to break down the idea:

- **What it does:** The memory system holds onto the instructions (like a recipe) and the data (like the ingredients) that the computer's processor (the part that does the thinking and calculating) needs to do its job.

- **Why it's important:** Without memory, the computer wouldn't be able to remember what it's supposed to do or what information it's using, making it impossible to run programs or perform tasks.

Learning Objectives

This part of our learning journey is all about what you'll be able to do by the time we're finished.

Here's what we'll cover:

- **Understanding Computer Memory:** We'll explore the main features of how computers store information. Think of it like understanding the different kinds of drawers and shelves in your filing cabinet – some are faster to access, some hold more, and they all work together.
- **Memory Hierarchy:** This is like organizing your filing cabinet. You'll have things you use all the time right on your desk (fastest), things you use often in a nearby drawer (medium speed), and things you rarely need stored away in a box in the basement (slowest). This system helps the computer get to information quickly.

Cache Memory: *We'll dive into a special, super-fast type of memory called cache memory. Imagine having a small, super-organized notepad right next to your desk where you jot down the most important pieces of information you're working with right now*. That's kind of what cache memory does for the computer's processor.*

- **Cache Design:** We'll look at the important decisions that go into setting up this fast notepad – how big it should be, and how it decides what information to keep handy.

Outline

We've seen how computer architecture is like the blueprint for a house, and memory is like a filing cabinet for the computer's brain (the processor). Now, let's look at what we'll be covering to understand how memory works, especially something called the 'memory hierarchy' and the specifics of 'cache memory'.

Here's a peek at what's coming up:

- **Cache Memory Principles:** We'll dive into the ideas behind cache memory, which is a super-fast, small memory that helps the processor get information quickly.
- **Elements of Cache Design:** We'll explore the different parts and decisions that go into creating this fast cache memory.
- **Organization, DRAM and SRAM:** This is about how memory is arranged and the two main types of computer memory: DRAM (Dynamic Random-Access Memory) and SRAM (Static Random-Access Memory). Think of DRAM like a whiteboard where you have to keep refreshing the writing to see it, and SRAM like a light switch that stays on or off until you flip it. They have different speeds and uses.
- **The Memory Hierarchy:** This is a big concept! Imagine you need a tool. You might have it right on your workbench (super fast, small storage), in a toolbox nearby (a bit slower, medium storage), or in a shed out back (slowest, largest storage). The memory hierarchy is about how computers use different types of memory, from the fastest to the slowest, to keep frequently used information close to the processor.

- **The Memory System:** We'll look at the overall setup of how all the memory components work together.
- **Interleaved Memory:** This is a technique to make memory access faster by spreading out memory across different locations so the computer can grab bits of information from multiple places at once, like having multiple cashiers at a supermarket checkout.
- **Memory Characteristics:** We'll discuss the different qualities of memory, like how fast it is, how much it can hold, and how much it costs.
- **Semiconductor Main Memory:** This refers to the main memory in computers, which is built using tiny electronic components on silicon chips.
- **Types of ROM:** We'll cover different kinds of ROM (Read-Only Memory), which is memory that holds information that can't be changed, like instructions that tell a device how to start up.
- **Chip Logic:** This refers to the internal workings and decision-making capabilities of memory chips.
- **Chip Packaging:** This is about how the actual memory chips are protected and connected to the rest of the computer system.

3.1 Memory Characteristics

Just like how we think about different types of storage for our belongings – maybe a handy shelf for things we use often, a bigger cupboard for less-used items, and a long-term storage unit for things we rarely need – computer memory also has different characteristics that describe how it works.

Think about how you might look for a specific book in a library. If the books are just piled randomly, it would take a very long time to find what you want. But if they are organized on shelves with clear labels, it's much faster. This brings us to a key characteristic: the **Method of Accessing**.

- **Sequential Access:** Imagine a very old-fashioned tape recorder. To get to a specific song, you have to fast-forward or rewind through all the songs before it. This is like sequential access, where you have to go through data in a specific order, one piece after another. Accessing a particular piece of information can take a long time because you might have to skip over a lot of other data.
- An example of this is tape units.
- **Direct Access:** This is a bit better than sequential access. Imagine a filing cabinet where each drawer has a label (like a general location), but within that drawer, the files might still be in order. You can quickly go to the right drawer, but then you might have to flip through a few files to find the exact one you need. This means access time can still vary.
- **Random Access:** This is like having a library where every single book has its own unique shelf number, and you can go directly to that shelf and pull out the book. In random access memory, any piece of data can be accessed directly, and the time it takes to get to any piece of data is always the same, no matter what you accessed before.
- This is the type of access used in main memory.
- **Associative Access:** This is a special kind of random access. Instead of looking for a book by its shelf number (address), imagine you could tell the librarian, "Give me any book that has the word 'dragon' on its cover." The system searches through all the books simultaneously to find one that matches a part of your description (a portion of its contents) rather than its exact location. Cache memories sometimes use this method.

Another important characteristic is **Capacity**, which is simply how much data a memory unit can hold. Think of it like the size of your filing cabinet – how many folders can it store? For computer memory, this is usually measured in **Bytes** (which are made up of 8 **bits**, the smallest unit of data).

We also need to consider **Unit of Transfer**. For main memory, this is the amount of data that the computer can read or write at one time. It's like how many pages you can pull out of a printer at once. Sometimes this is just a small chunk, and sometimes it's a much larger chunk called a **block**, especially for external storage like hard drives.

Performance is another big one. How quickly can we get information in and out of memory?

- **Access Time (latency):** This is how long it takes to get a piece of data. For random access memory, it's the time from when the computer asks for data to when it's ready. For other types, it's the time it takes to get the read/write mechanism to the right spot.

Memory Cycle Time: *This is the total time needed for one memory operation, including any extra time needed before the next* operation can start.* This is important for how fast the computer can keep asking for data.

- **Transfer Rate:** This is how quickly data can be moved into or out of memory, measured in bits per second (bps). It's like the speed of a conveyor belt moving data.

Finally, there are **Physical Characteristics**:

- **Volatile Memory:** This is like a whiteboard. When you turn off the power (erase it), everything is gone. The information disappears when the power is cut.
- **Nonvolatile Memory:** This is like writing on paper with a pen. The information stays there even when you're not actively doing anything or when the power is off, until you deliberately change it.

Computer Memory

We've already touched on how computer memory is like a filing cabinet for the computer, storing instructions and data. This section dives a little deeper into what makes up that filing cabinet.

Think about how you organize your own things. You might have things you use all the time right on your desk (super fast to grab!), other things in a nearby drawer (pretty quick), and then things you only need occasionally stored away in a closet or attic (takes longer to get to).

Computer memory works similarly! It's a system designed to store information that the computer needs to use right away. When people casually talk about 'memory' in a computer, they usually mean the 'primary storage' or 'main memory' – the stuff the computer actively works with.

Computer memory is a super important part of the computer's brain (the CPU, or Central Processing Unit). While the idea of memory might seem simple, there are actually many different types, and they can be sorted in a few ways:

- **Technology:** What they're made of.
- **Organization:** How the information is arranged.
- **Performance:** How fast they are.

- **Cost:** How much they cost.

Because computers have different needs, no single type of memory works for everything. This is why most computers have a 'hierarchy of memory subsystems'. Imagine having different storage locations:

- **Internal memories:** These are like your desk or nearby drawers. They are directly accessible by the CPU. Examples include processor registers (tiny, super-fast storage right inside the CPU) and cache memory (a small, fast memory that holds frequently used data).
- **External memories:** These are like your closet or attic. They are not directly accessed by the CPU but through special input/output (I/O) modules. Examples include things like optical disks (like CDs or DVDs) and magnetic disks (like hard drives).

Table 1, which we'll look at, lists out all these different characteristics that describe computer memory.

3.2 The Memory Hierarchy

Imagine you're trying to manage all your belongings. You might have a few very special items you keep right on your person because you use them constantly (like your keys or wallet). Then, you'd have a small set of frequently used items in a nearby drawer, and a larger collection of things in your closet, and finally, a huge storage unit for everything else you rarely need.

Computers face a similar challenge with storing information. There's a trade-off between how much information a memory can hold (its **capacity**), how quickly it can be accessed (its **access time**), and how much it costs to make (its **cost per bit**).

Generally, the relationships are:

- **Faster access time** usually means a **greater cost per bit**.
- **Greater capacity** usually means a **slower access time**.
- **Greater capacity** also usually means a **smaller cost per bit**.

This creates a dilemma for computer designers. They want lots of storage (high capacity) because programs and data are getting bigger, and they want it to be cheap (low cost per bit). But to make the computer run fast, they need quick access to information, which means using expensive, smaller memories with short access times.

The solution is to use a **memory hierarchy**. This is like having different levels of storage, each with different characteristics, organized to work together efficiently. As you move down this hierarchy (from fastest to slowest), the following happens:

- **Access time increases** (it takes longer to get information).
- **Capacity increases** (you can store more information).
- **Cost per bit decreases** (it's cheaper to store each piece of information).
- **Frequency of access decreases** (the processor needs to get information from this level less often).

The very top of this hierarchy, the fastest and smallest memory, consists of **registers**. These are tiny storage locations directly inside the computer's main processing unit (the **processor**), which is like the computer's brain. They hold a very small amount of data that the processor is working on *right now*.

Next comes **main memory**, also known as **RAM** (Random Access Memory). This is the computer's primary working storage. Think of it like a large desk where you keep the documents and tools you're actively using for a project. Each spot in main memory has a unique address, like a house number, so the processor can find things.

To speed things up even more, there's usually a **cache memory** between the processor and main memory. The cache is a small, very fast memory that stores copies of data from main memory that the processor is likely to need soon. It's like having a small, super-fast notepad next to your desk where you jot down the most important pieces of information you'll need in the next few minutes. The cache isn't usually something a programmer or even the processor directly thinks about; it works automatically to improve performance.

Registers, cache, and main memory are typically **volatile**, meaning they lose their stored information when the power is turned off. They use **semiconductor technology**, which is a way of making electronic components on silicon chips.

Below main memory are **external mass storage devices**. These are slower but much larger and cheaper than main memory, and they are usually **nonvolatile**, meaning they keep their data even when the power is off. Examples include hard disk drives and removable media like USB drives or optical discs (CDs/DVDs). This is like your filing cabinets or storage boxes for things you don't need immediately but want to keep long-term. This external storage is often referred to as **secondary memory** or **auxiliary memory** and is used to store program and data files. It can even be used to create **virtual memory**, which makes the computer seem like it has more main memory than it actually does by using disk space as an extension.

3.3 Cache Memory

Imagine you're a super-fast chef (the CPU) working in a kitchen. You need ingredients (data) and recipes (instructions) to cook. The main pantry (main memory) is huge but takes a while to walk to. To speed things up, you have a small, super-close countertop right next to you where you keep the ingredients and recipe pages you're using *right now*. This is like **cache memory**.

Cache memory is a very fast type of computer memory that acts as a temporary holding area, or **buffer**, between the main memory (like your big pantry) and the CPU (the chef). It's designed to hold copies of data and instructions that are used frequently, so the CPU can grab them much faster.

Here's how it works:

- **Speed vs. Size Trade-off:** Cache memory is much faster than main memory, but it's also more expensive and has a smaller capacity. It's a way to get the best of both worlds: the speed of expensive, fast memory and the larger size of less expensive, slower memory.
- **How it's Used:** When the CPU needs a piece of data or an instruction, it first checks the cache.
- **Cache Hit:** If the item is found in the cache, it's delivered to the CPU immediately. This is like finding the ingredient on your countertop – super fast!

Cache Miss: *If the item is not** in the cache, the computer has to go to the slower main memory to get it. But here's the clever part: when it fetches data from main memory, it doesn't just get that one piece; it brings a whole chunk of related data (called a **block**) into the cache. This is like grabbing a whole shelf of

related spices from the pantry because you know you'll likely need others soon.

- **Why it Works (Locality of Reference):** This strategy is effective because programs tend to access the same data or instructions, or data located very close to what they just accessed, over and over again. This tendency is called **locality of reference**.

- **Cache Structure:**

- Main memory is thought of as being divided into **blocks** (groups of data).
- The cache is divided into **lines**, and each line can hold one block from main memory.
- Because there are usually many more blocks in main memory than there are lines in the cache, a single line in the cache can't be permanently assigned to just one block.
- To know which block is currently stored in a cache line, each line has a **tag**. This tag is like a label that tells you which specific block from main memory is currently residing in that cache line.
- **Levels of Cache:** Some systems have multiple levels of cache (e.g., L1, L2, L3). Think of it like having a tiny, super-fast shelf right next to the stove (L1 cache), a slightly larger, still fast countertop (L2 cache), and a bigger, less fast rolling cart (L3 cache) before you have to go to the main pantry (main memory). Each level is generally slower and larger than the one before it.
- **Cost:** Cache memory is more expensive per unit than main memory or disk storage, but it's cheaper than the very fastest storage, like CPU registers (which are like the chef's hands, holding only one or two things at a time).

3.3.1 Cache Performance

Imagine you're constantly needing to look up facts for a project. You have a big library (main memory) but it takes a while to find things. You also have a small desk right next to you (cache memory) where you keep the books and notes you're using most often.

When the computer's processor (the part that does the thinking and calculations) needs some information, it first checks that handy desk (the cache).

- If the information is found on the desk, it's called a **cache hit**. This is great because it's super fast!
- If the information isn't on the desk, it's called a **cache miss**. The processor then has to go to the big library (main memory) to get it. When this happens, a fresh copy of that information is brought to the desk, and the desk might rearrange itself to make sure the most-used items are easily accessible.

We measure how well the cache is working using something called the **hit ratio**.

- The **hit ratio** is basically the percentage of times the processor found what it needed on the desk (cache) versus how many times it had to go to the library (main memory). It's calculated as: $\text{hit ratio} = \text{number of hits} / (\text{number of hits} + \text{number of misses})$.

To make the cache work even better, we can do a few things:

- Make the cache desk bigger or better organized.
- Try to reduce the number of times we have a cache miss.
- Reduce the time it takes to get information from the library when we do have a miss.
- Speed up the time it takes to find something on the desk when we have a hit.

3.3.2 Cache Read operation

Imagine you're looking for a specific tool in your workshop. You first check your most accessible workbench (the cache). If the tool is there, great! You grab it quickly.

When the computer's processor (the brain that does the work) needs a piece of information (a 'word'), it first checks the cache. This 'word' has a specific address, like a shelf number.

- **Cache Hit:** If the requested word is found in the cache, it's called a 'cache hit'. The cache quickly hands this word over to the processor.
- **Cache Miss:** If the word isn't in the cache, it's a 'cache miss'. The computer then needs to go to the main memory (like a bigger storage room) to find the block of information that contains the word you need.

In many modern computers, when there's a cache miss, the computer can fetch the needed block from main memory and put it into the cache at the same time it sends the specific word to the processor. This is like going to the big storage room, grabbing the whole shelf of tools you need, and handing the one specific tool to your assistant while also putting the shelf back in your workshop.

How the computer is wired for this:

- The cache and the processor are connected by lines for sending data, giving commands (control lines), and specifying addresses (address lines).
- These data and address lines also connect to 'buffers'. Buffers are like small holding areas that can temporarily store data or addresses before they move on.
- These buffers connect to a 'system bus'. A system bus is like a main highway that connects different parts of the computer, including main memory.

What happens during a read:

- **Cache Hit:** If the word is in the cache (a hit), the buffers are essentially turned off, and all communication happens directly between the processor and the cache. No traffic goes onto the main system bus.
- **Cache Miss:** If the word is NOT in the cache (a miss), the address of the data is put onto the system bus. The data is then returned through the data buffer to both the cache (to be stored there for future use) and directly to the processor.

Sometimes, the cache is placed physically *between* the processor and main memory for all data, address, and control signals. In this setup, if there's a cache miss, the word is first brought into the cache and *then* moved from the cache to the processor.

3) Mapping Function

Imagine your cache memory is a small set of special, super-fast shelves, and your main memory is a huge warehouse. Since you have way more items in the warehouse than you can fit on those fast shelves, you need a system to decide which items from the warehouse go onto which shelf.

This system is called a **mapping function**. It's basically a rulebook that tells the computer:

1. How to decide which items (called **blocks** in main memory) can be placed on the available shelves (called **lines** in cache memory).
2. How to know which specific item from the warehouse is currently on each shelf.

The choice of this rulebook (the mapping function) is what determines how the cache is organized and how it works. There are three main ways to do this:

Direct Mapping: *This is like having a designated spot for each warehouse item. If a certain item from the warehouse needs to go on a shelf, it can only* go in one specific spot. If that spot is already taken, the old item has to be thrown out to make room.*

Associative Mapping: *This is more flexible. Any item from the warehouse can go on any* empty shelf. The computer just needs to remember where it put it.*

Set-Associative Mapping: *This is a mix of the two. Imagine your shelves are grouped into sections (called **sets**). An item from the warehouse can go into any shelf within a specific* section, but not just anywhere. It's like having a few possible spots for each item, rather than just one or any spot at all.*

1) Cache Addresses

Imagine your computer has a special way of organizing its memory, like a librarian who can make sure you always have access to the books you need, even if the library itself is much bigger than what you can physically hold.

This system is called **virtual memory**. It lets programs act as if they have a huge amount of memory available, even if the computer's actual physical memory is smaller. When a program is running, the addresses it uses are **virtual addresses**. These are like the shelf numbers a librarian might use, which are logical references within the library's system, rather than the actual physical location of the book on a specific shelf.

Almost all modern processors, and even many smaller ones found in everyday devices, use this virtual memory system. It's a clever way to manage memory resources efficiently.

3.3.3 Elements of Cache Design

While there are many ways to build a cache memory, most of them can be understood by looking at a few core design choices. These choices help us describe and tell apart different cache systems.

Think of designing a special express lane for a busy highway. You have to decide a few things:

- **How do cars get into the express lane?** (This relates to how data from the main memory gets into the cache).
- **Where in the express lane can a car go?** (This is about where a piece of data can be stored in the cache).
- **How do you find a car (data) once it's in the express lane?** (This is about how the cache is organized to quickly locate requested information).

These fundamental design elements are what we'll explore to understand how different caches work. Sometimes, we'll see how these designs are used in super-powerful computers designed for complex scientific tasks (often called high-performance computing or HPC).

2) Cache Size

We want our cache to be just the right size. Imagine you have a toolbox for your tools. You want it big enough to hold the tools you use most often, so you can grab them quickly, but not so big that it becomes cumbersome to search through or expensive to buy.

- **Goal 1: Cost Balance:** We want the cache to be small enough so that the average cost of storing information across both the cache and the main memory is similar to the cost of just having the main memory.
- **Goal 2: Speed Balance:** We also want the cache to be large enough so that when the processor needs something, it's usually found there, making the average time to access information very close to the fast time of the cache itself.
- **Other Reasons to Keep it Small:** Making the cache larger means more complex electronic circuits (called 'gates') are needed just to figure out how to access the information stored within it. This adds to the complexity and potentially the cost.

5) Write Policy

Imagine you're editing a document on your computer. Sometimes, you make changes and save them, and sometimes you might close the document without saving. This section is about how the computer decides what to do with changes made in the super-fast cache memory when that information needs to be moved or replaced.

When the computer needs to swap out an old piece of information (called a 'block') from the cache to make room for new information, it has two main scenarios:

- **No changes made:** If the old information in the cache hasn't been changed at all since it was brought in, the computer can just toss it out and bring in the new information without any fuss. It's like throwing away an old, unused note.
- **Changes were made:** If even one small part of the old information in the cache has been changed (this is called a 'write operation'), the computer needs to be careful. Before it can bring in the new information, it must first update the main memory with the modified old information. This is like making sure you've saved your edits to a document before closing it, so you don't lose your work.

There are different ways (called 'write policies') the computer can handle these 'write' situations, and each has its own pros and cons regarding speed and cost. We'll explore these more.

4) Replacement Algorithms

Imagine your cache memory is a small whiteboard where the processor (the computer's brain) writes down frequently used notes. When the whiteboard gets full and the processor needs to write a new note, it has to erase an old one. A 'replacement algorithm' is simply the rule the computer uses to decide WHICH old

note to erase.

Different ways the computer decides which 'note' (or 'block' of data) to erase from the cache when it's full:

- **Least Recently Used (LRU):** This is like erasing the note that hasn't been looked at for the longest time. It's generally the most effective strategy.
- **First-In, First-Out (FIFO):** This is like erasing the note that was written on the whiteboard first. The oldest note gets erased, no matter if it's still being used.
- **Least Frequently Used (LFU):** This is like erasing the note that has been used the fewest times.
- **Random Replacement (RR):** This is like randomly picking a note to erase.

For simple 'direct mapping' (where there's only one spot for each piece of data), there's no choice – the spot is fixed. But for more flexible ways of organizing the cache ('associative' and 'set-associative' mapping), the computer needs one of these algorithms to make a decision about what to remove. To keep things fast, these decisions are made by the computer's hardware.

7) Number of Caches

Imagine you have a super-fast notepad right next to you for your most important to-do items, and a larger filing cabinet further away for everything else. This helps you get things done much quicker.

Computers can use a similar idea with their memory. When computers first started using these "quick notepads" (called caches), they usually only had one. But now, it's common to have several levels of these quick notepads, organized in a smart way.

This idea of having multiple levels of caches helps the computer's main processing unit (the CPU) run even faster. When a cache is built right onto the same computer chip as the CPU, it's called an **on-chip cache**. This is super fast because the CPU doesn't have to go out to a separate, slower part of the computer to get information. It's like having that notepad *right* in your hand instead of on a desk across the room.

Having multiple caches means we need to think about two main things:

- **How many levels of caches** there are (like having a tiny, super-fast notepad, a slightly bigger, still-fast notepad, and then a main notebook).
- Whether these caches store instructions and data together (**unified caches**) or separately (**split caches**).

6) Line size

Imagine you're baking and need a specific spice. Instead of just grabbing that one spice jar, you decide to grab a whole shelf of spices that are located near it, just in case you need them soon. This is similar to how computers work with 'line size' in cache memory.

When the computer needs a piece of information (called a 'word') from main memory and brings it into the cache, it doesn't just bring that single word. It also brings along some neighboring words. This group of words brought together is called a 'block' or a 'line'.

Why do this? It's based on a principle called 'locality of reference'. This fancy term means that if the computer uses a piece of data, it's very likely to use data that's located close to it very soon. So, bringing in a whole block of nearby data increases the chance that the next piece of information the processor needs will already be in the cache.

- **Small blocks:** Bringing in small blocks is good because it's more likely that all the data in that small block will be used, improving the 'hit ratio' (how often the processor finds what it needs in the cache).

Large blocks: *However, if the block gets too big, the computer might end up bringing in a lot of data it won't actually use. This means some useful data that was* in the cache might have to be thrown out to make space for the new, potentially unused data. This can actually decrease the hit ratio.*